Assignment 3

maitmad

2026-10-14

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com (http://rmarkdown.rstudio.com).

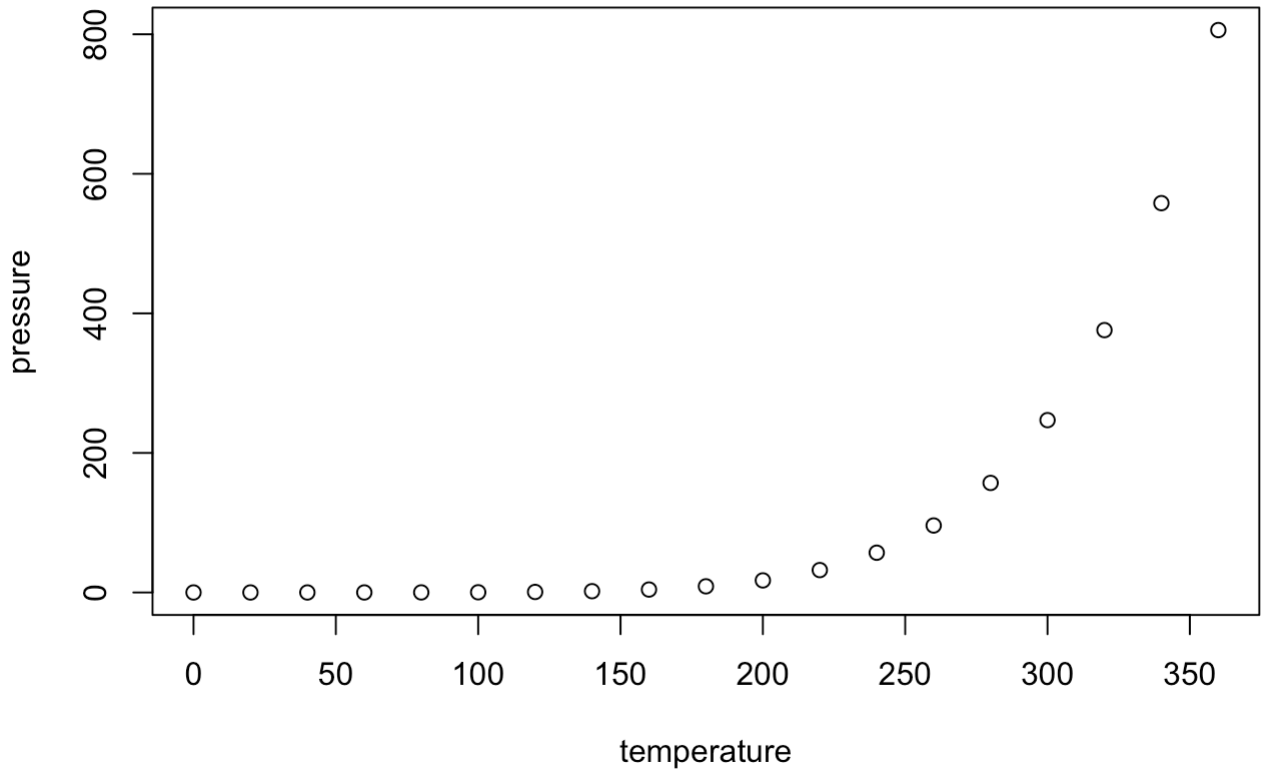
When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00
```

Including Plots

You can also embed plots, for example:



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.5.2
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.5.2
```

```
setwd("~/Desktop/KSU/Sem 1/Machine Learning/ass 3")
```

```
getwd()
```

```
## [1] "/Users/muhammadbilalaitmad/Desktop/KSU/Sem 1/Machine Learning/ass 3"
```

```
UniversalBank <- read.csv("UniversalBank.csv")
```

Now that we have the data, we can inspect it

```
str(UniversalBank)
```

```
## 'data.frame':    5000 obs. of  14 variables:
##  $ ID                : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ Age               : int  25 45 39 35 35 37 53 50 35 34 ...
##  $ Experience        : int  1 19 15 9 8 13 27 24 10 9 ...
##  $ Income            : int  49 34 11 100 45 29 72 22 81 180 ...
##  $ ZIP.Code          : int  91107 90089 94720 94112 91330 92121 91711 93943 90089
93023 ...
##  $ Family            : int  4 3 1 1 4 4 2 1 3 1 ...
##  $ CCAvg             : num  1.6 1.5 1 2.7 1 0.4 1.5 0.3 0.6 8.9 ...
##  $ Education         : int  1 1 1 2 2 2 2 3 2 3 ...
##  $ Mortgage          : int  0 0 0 0 0 155 0 0 104 0 ...
##  $ Personal.Loan     : int  0 0 0 0 0 0 0 0 0 1 ...
##  $ Securities.Account: int  1 1 0 0 0 0 0 0 0 0 ...
##  $ CD.Account        : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ Online            : int  0 0 0 0 0 1 1 0 1 0 ...
##  $ CreditCard        : int  0 0 0 0 1 0 0 1 0 0 ...
```

```
summary(UniversalBank)
```

```
##        ID            Age          Experience       Income          ZIP.Code
##  Min.   :   1   Min.   :23.00   Min.   :-3.0   Min.   :  8.00   Min.   : 9307
##  1st Qu.:1251   1st Qu.:35.00   1st Qu.:10.0   1st Qu.: 39.00   1st Qu.:91911
##  Median :2500   Median :45.00   Median :20.0   Median : 64.00   Median :93437
##  Mean   :2500   Mean   :45.34   Mean   :20.1   Mean   : 73.77   Mean   :93153
##  3rd Qu.:3750   3rd Qu.:55.00   3rd Qu.:30.0   3rd Qu.: 98.00   3rd Qu.:94608
##  Max.   :5000   Max.   :67.00   Max.   :43.0   Max.   :224.00   Max.   :96651
##      Family          CCAvg          Education        Mortgage
##  Min.   :1.000   Min.   : 0.000   Min.   :1.000   Min.   :  0.0
##  1st Qu.:1.000   1st Qu.: 0.700   1st Qu.:1.000   1st Qu.:  0.0
##  Median :2.000   Median : 1.500   Median :2.000   Median :  0.0
##  Mean   :2.396   Mean   : 1.938   Mean   :1.881   Mean   : 56.5
##  3rd Qu.:3.000   3rd Qu.: 2.500   3rd Qu.:3.000   3rd Qu.:101.0
##  Max.   :4.000   Max.   :10.000   Max.   :3.000   Max.   :635.0
##  Personal.Loan   Securities.Account   CD.Account         Online
##  Min.   :0.000   Min.   :0.0000     Min.   :0.0000   Min.   :0.0000
##  1st Qu.:0.000   1st Qu.:0.0000     1st Qu.:0.0000   1st Qu.:0.0000
##  Median :0.000   Median :0.0000     Median :0.0000   Median :1.0000
##  Mean   :0.096   Mean   :0.1044     Mean   :0.0604   Mean   :0.5968
##  3rd Qu.:0.000   3rd Qu.:0.0000     3rd Qu.:0.0000   3rd Qu.:1.0000
##  Max.   :1.000   Max.   :1.0000     Max.   :1.0000   Max.   :1.0000
##    CreditCard
##  Min.   :0.000
##  1st Qu.:0.000
##  Median :0.000
##  Mean   :0.294
##  3rd Qu.:1.000
##  Max.   :1.000
```

Perfect, now let's split!

```
set.seed(123)

trainIndex <- createDataPartition(
  UniversalBank$Personal.Loan,
  p = 0.60,
  list = FALSE
)

train <- UniversalBank[trainIndex, ]
valid <- UniversalBank[-trainIndex, ]
```

Checking sizes

```
nrow(train)
```

```
## [1] 3000
```

```
nrow(valid)
```

```
## [1] 2000
```

Size validated

```
trainNB <- train %>%
  select(Online, CreditCard, Personal.Loan)

names(trainNB) <- c("Online", "CC", "Loan")

pivot_A <- table(trainNB$CC,
                 trainNB$Loan,
                 trainNB$Online)

pivot_A
```

```
## , ,  = 0
##
##
##        0    1
##   0  785   65
##   1  317   34
##
## , ,  = 1
##
##
##        0    1
##   0 1145  122
##   1  475   57
```

```
ftable(pivot_A)
```

```
##         0    1
##
## 0 0   785 1145
##   1    65  122
## 1 0   317  475
##   1    34   57
```

Probability of Accepting Loan Given CC = 1 and Online = 1

```
subset_counts <- table(
  trainNB$Loan[trainNB$CC == 1 &
               trainNB$Online == 1]
)

subset_counts
```

```
##
##   0   1
## 475  57
```

```
## Computing probability

prob_B <- subset_counts["1"] / sum(subset_counts)

prob_B
```

```
##         1
## 0.1071429
```

Loan as a function of online

```
pivot_online <- table(trainNB$Loan,
                      trainNB$Online)

pivot_online
```

```
##
##        0    1
##   0 1102 1620
##   1   99  179
```

Loan as a function of CC

```
pivot_cc <- table(trainNB$Loan,
                  trainNB$CC)

pivot_cc
```

```
##
##        0    1
##   0 1930  792
##   1  187   91
```

P(CC = 1 | Loan = 1)

```
p_cc1_loan1 <-
  sum(trainNB$CC == 1 &
      trainNB$Loan == 1) /
  sum(trainNB$Loan == 1)

p_cc1_loan1
```

```
## [1] 0.3273381
```

P(Online = 1 | Loan = 1)

```
p_online1_loan1 <-
  sum(trainNB$Online == 1 &
      trainNB$Loan == 1) /
  sum(trainNB$Loan == 1)

p_online1_loan1
```

```
## [1] 0.6438849
```

P(Loan = 1)

```
p_loan1 <- mean(trainNB$Loan == 1)

p_loan1
```

```
## [1] 0.09266667
```

P(CC = 1 | Loan = 0)

```
p_cc1_loan0 <-
  sum(trainNB$CC == 1 &
      trainNB$Loan == 0) /
  sum(trainNB$Loan == 0)

p_cc1_loan0
```

```
## [1] 0.2909625
```

P(Online = 1 | Loan = 0)

```
p_online1_loan0 <-
  sum(trainNB$Online == 1 &
      trainNB$Loan == 0) /
  sum(trainNB$Loan == 0)

p_online1_loan0
```

```
## [1] 0.5951506
```

P(Loan = 0)

```
p_loan0 <- mean(trainNB$Loan == 0)

p_loan0
```

```
## [1] 0.9073333
```

Compute Conditional Probability

```
num <-
  p_cc1_loan1 *
  p_online1_loan1 *
  p_loan1

den <-
  num +
  p_cc1_loan0 *
  p_online1_loan0 *
  p_loan0

prob_NB <- num / den

prob_NB
```

```
## [1] 0.1105637
```

Comparing with part B

```
prob_B
```

```
##         1
## 0.1071429
```

```
prob_NB
```

```
## [1] 0.1105637
```

The pivot-table estimate (0.107) is the empirical conditional probability observed directly in the training data and therefore does not rely on any modeling assumptions. The Naive Bayes estimate (0.111) assumes conditional independence between Credit Card ownership and Online Banking usage given loan status. Although this assumption may not hold exactly, the resulting probability is very close to the empirical estimate. Given the relatively large training sample (3000 observations) and the substantial number of customers in the CC = 1 and Online = 1 subgroup, the direct pivot-table estimate is likely the more accurate estimate of the true conditional probability. However, Naive Bayes remains an attractive classification method because it requires little computational power, does not require normalization of predictors, involves minimal hyperparameter tuning, and often performs well even when the independence assumption is not perfectly satisfied.

Convert to factors

```
trainNB$Loan <- factor(trainNB$Loan)
trainNB$CC <- factor(trainNB$CC)
trainNB$Online <- factor(trainNB$Online)
```

Fit Naive Bayes

```
nb_model <- naiveBayes(
  Loan ~ CC + Online,
  data = trainNB
)

nb_model
```

```
##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##          0          1
## 0.90733333 0.09266667
##
## Conditional probabilities:
##    CC
## Y           0         1
##   0 0.7090375 0.2909625
##   1 0.6726619 0.3273381
##
##    Online
## Y           0         1
##   0 0.4048494 0.5951506
##   1 0.3561151 0.6438849
```

Laplace Smoothing

```
nb_model_laplace <- naiveBayes(
  Loan ~ CC + Online,
  data = trainNB,
  laplace = 1
)

nb_model_laplace
```

```
##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##          0          1
## 0.90733333 0.09266667
##
## Conditional probabilities:
##    CC
## Y           0         1
##   0 0.7088840 0.2911160
##   1 0.6714286 0.3285714
##
##    Online
## Y           0         1
##   0 0.4049192 0.5950808
##   1 0.3571429 0.6428571
```

Predict for Customer with CC = 1 and Online = 1

```
new_customer <- data.frame(
  CC = factor(1, levels = c(0,1)),
  Online = factor(1, levels = c(0,1))
)

predict(
  nb_model_laplace,
  new_customer,
  type = "raw"
)
```

```
##              0         1
## [1,] 0.8892638 0.1107362
```

The Naive Bayes model was fit using the predictors Credit Card (CC) and Online Banking (Online). The model estimates the posterior probability P(Loan=1∣CC=1,Online=1) by combining the prior probability of loan acceptance with the conditional probabilities of the predictor values given the loan outcome.

The manually calculated Naive Bayes probability from Part E was approximately 0.111. The probability produced by the Naive Bayes model for a customer with CC = 1 and Online = 1 should be nearly identical, with any small differences attributable to rounding or smoothing procedures. This agreement confirms that the manual calculations were performed correctly and that the software implementation is applying the same Naive Bayes formula.

Comparing this result with the direct pivot-table estimate from Part B (0.107) shows that the two probabilities are very similar. This suggests that the conditional independence assumption made by Naive Bayes does not substantially distort the estimate for these variables. Because the training set contains approximately 3,000 observations, the direct conditional probability from the pivot table is likely a reliable estimate. However, Naive

Bayes remains valuable because it is computationally efficient, requires no feature normalization, involves little to no hyperparameter tuning, and can generalize well even when the independence assumption is not perfectly satisfied.